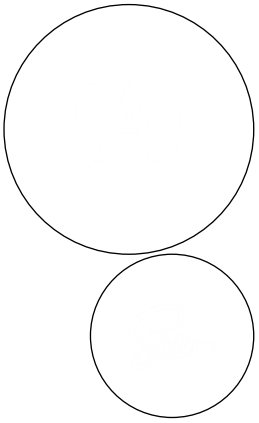
Desarrollo Web Integrado · 100000ST61

Fundamentos de Angular

Laboratorio dirigido.



TechStore Web: catálogo con Angular y SASS

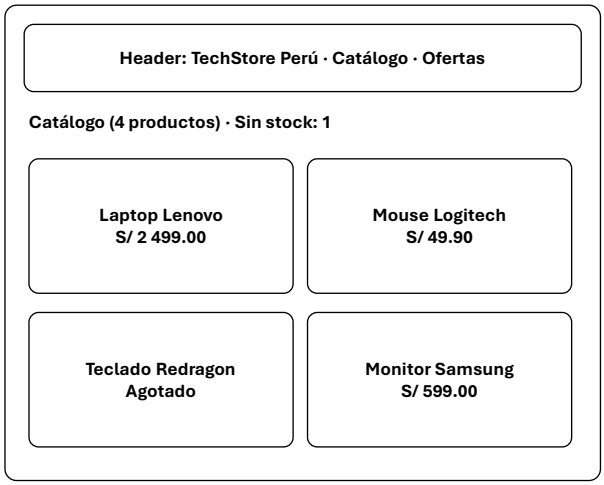
Unidad 3 · Semana 11 · Sesión 11

Mag. Ing. Marco Antonio Flores Torres · UTP Lima Sur

1

LAB 11 · PRESENTACIÓN

Laboratorio: catálogo TechStore Web



Escenario

TechStore Perú SAC ya tiene su API en Spring Boot (pe.utp.techstore). Construye la interfaz Angular de su catálogo con componentes y estilos SASS.

- **Duración:** 60 a 90 minutos, en parejas.
- **Entorno:** Node.js reciente, Angular CLI y VS Code.
- **Resultado:** app en localhost:4200 con header y 4 tarjetas.
- **Entregable:** carpeta del proyecto (sin node_modules) y captura.

2

LAB 11 · PASO 1

Paso 1: crear y ejecutar el proyecto

Terminal

```
# 1. Verifica tu entorno
node -v
npm -v

# 2. Instala Angular CLI (si no lo tienes)
npm install -g @angular/cli

# 3. Crea el proyecto con SCSS
ng new techstore-web --style=scss --ssr=false

# 4. Entra y ejecuta
cd techstore-web
ng serve

# 5. Abre http://localhost:4200
```

Verifica

Debes ver la página de bienvenida de Angular en el navegador y ningún error rojo en la terminal.

Si el puerto está ocupado

Usa ng serve --port 4300 y abre localhost:4300. Si el CLI ofrece herramientas de IA, elige ninguna.

3

LAB 11 · PASO 2

Paso 2: prepara la estructura SASS

src/scss/_variables.scss

```
$color-primario: #dd0031;
$color-secundario: #2b1e3a;
$color-fondo: #f6f1f4;
$color-exito: #1f8a70;
$color-alerta: #f2a900;
$radio: 12px;
$espacio: 16px;
$fuente: 'Segoe UI', Roboto, Arial, sans-serif;
```

src/styles.scss

```
@use './scss/variables' as v;

body {
  margin: 0;
  font-family: v.$fuente;
  background: v.$color-fondo;
  color: v.$color-secundario;
}
```

src/scss/_mixins.scss

```
@use 'variables' as v;

@mixin tarjeta($padding: 16px) {
  background: #fff;
  border-radius: v.$radio;
  padding: $padding;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.12);
}

@mixin badge($fondo, $texto: #fff) {
  display: inline-block;
  padding: 2px 10px;
  border-radius: 999px;
  background: $fondo;
  color: $texto;
  font-size: 0.8rem;
}

@mixin desde($ancho) {
  @media (min-width: $ancho) { @content; }
}
```


4

LAB 11 · PASOS 3 Y 4

Pasos 3 y 4: modelo de datos y componentes

Paso 3 · src/app/models/producto.model.ts

```
// Espejo de la entidad Producto de Spring Boot
export interface Producto {
  id: number;
  nombre: string;
  categoria: string;
  precio: number;
  stock: number;
}
```

Paso 4 · Terminal

```
ng g c components/header
ng g c components/product-card
ng g c components/product-list
```

Por qué una interface

Describe la forma del JSON de tu API. Si el nombre de un campo no coincide, el editor te avisa antes de ejecutar.

Revisa

Deben existir tres carpetas dentro de src/app/components, cada una con sus archivos .ts, .html y .scss.

LAB 11 · PASO 5

Paso 5: componente Header

header.html

```
<header class="header">
  <h1 class="header__logo">
    TechStore <span>Perú</span>
  </h1>
  <nav class="header__nav">
    <a href="#">Catálogo</a>
    <a href="#">Ofertas</a>
    <a href="#">Contacto</a>
  </nav>
</header>
```

Nota

header.ts se deja como lo generó el CLI: este componente no necesita lógica.

header.scss

```
@use '../../../scss/variables' as v;
@use '../../../scss/mixins' as m;

.header {
  background: v.$color-secundario;
  color: #fff;
  padding: v.$espacio;
  display: flex;
  flex-direction: column;
  gap: 8px;

  &__logo span { color: v.$color-primario; }
  &__nav {
    display: flex;
    gap: v.$espacio;
    a { color: #fff; text-decoration: none; }
  }

  @include m.desde(768px) {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }
}
```

LAB 11 · PASO 6

Paso 6: ProductCard (clase)

product-card.ts

```
import { Component, computed, input, signal }
  from '@angular/core';
import { CurrencyPipe } from '@angular/common';
import { Producto } from '../../models/producto.model';

@Component({
  selector: 'app-product-card',
  imports: [CurrencyPipe],
  templateUrl: './product-card.html',
  styleUrl: './product-card.scss'
})
export class ProductCard {
  producto = input.required<Producto>();
  agotado = computed(
    () => this.producto().stock === 0);
  agregados = signal(0);

  agregar() {
    this.agregados.update(n => n + 1);
  }
}
```

- **input.required:** el padre debe enviar el producto.
- **computed:** agotado se recalcula solo.
- **signal:** agregados es estado propio de la tarjeta.
- **CurrencyPipe:** formatea el precio en soles.

LAB 11 · PASO 7

Paso 7: ProductCard (plantilla)

product-card.html

```
<article class="card" [class.card--agotado]="agotado()">
  <span class="card__categoria">{{ producto().categoria }}</span>
  <h3 class="card__nombre">{{ producto().nombre }}</h3>
  <p class="card__precio">
    {{ producto().precio | currency:'PEN':'S/ ' }}
  </p>

  @if (agotado()) {
    <span class="badge badge--agotado">Agotado</span>
  } @else {
    <span class="badge badge--ok">Stock: {{ producto().stock }}</span>
  }

  <button type="button" [disabled]="agotado()" (click)="agregar()">
    Agregar ({{ agregados() }})
  </button>
</article>
```

LAB 11 · PASO 8

Paso 8: ProductCard (estilos SASS)

product-card.scss (parte 1)

```
@use 'sass:color';
@use '../../../scss/variables' as v;
@use '../../../scss/mixins' as m;

.card {
  @include m.tarjeta;
  display: flex;
  flex-direction: column;
  gap: 8px;

  &__categoria {
    color: v.$color-primario;
    text-transform: uppercase;
  }
  &__precio { font-weight: 700; }
  &--agotado { opacity: 0.6; }
}
```

product-card.scss (parte 2)

```
.card button {
  background: v.$color-primario;
  color: #fff;
  border: 0;
  border-radius: 8px;
  padding: 8px 12px;

  &:hover {
    background: color.adjust(
      v.$color-primario, $lightness: -8%);
  }
  &:disabled { background: #bbb; }
}

.badge {
  &--ok { @include m.badge(v.$color-exito); }
  &--agotado {
    @include m.badge(v.$color-alerta,
                     v.$color-secundario);
  }
}
```

Desarrollo Web Integrado · Sesión 11 · UTP 9

9

LAB 11 · PASO 9

Paso 9: ProductList (clase)

product-list.ts

```
import { Component, computed, signal } from '@angular/core';
import { ProductCard } from '../product-card/product-card';
import { Producto } from '../../models/producto.model';

@Component({
  selector: 'app-product-list',
  imports: [ProductCard],
  templateUrl: './product-list.html',
  styleUrl: './product-list.scss'
})
export class ProductList {
  productos = signal<Producto[]>([
    { id: 1, nombre: 'Laptop Lenovo IdeaPad 3', categoria: 'Laptops', precio: 2499, stock: 8 },
    { id: 2, nombre: 'Mouse Logitech M170', categoria: 'Accesorios', precio: 49.9, stock: 25 },
    { id: 3, nombre: 'Teclado Redragon Kumara', categoria: 'Accesorios', precio: 159, stock: 0 },
    { id: 4, nombre: 'Monitor Samsung 24 pulgadas', categoria: 'Monitores', precio: 599, stock: 5 },
  ]);
  sinStock = computed(() => this.productos().filter(p => p.stock === 0).length);
}
```

Desarrollo Web Integrado · Sesión 11 · UTP 10

10

LAB 11 · PASO 9

Paso 9: ProductList (plantilla y estilos)

product-list.html

```
<section class="lista">
  <h2>Catálogo ({{ productos().length }} productos)</h2>
  <p class="lista__aviso">Sin stock: {{ sinStock() }}</p>

  <div class="lista__grid">
    @for (p of productos(); track p.id) {
      <app-product-card [producto]="p" />
    } @empty {
      <p>No hay productos disponibles.</p>
    }
  </div>
</section>
```

product-list.scss

```
@use '../../../scss/variables' as v;

.lista {
  &__aviso {
    color: v.$color-primario;
    font-weight: 700;
  }
  &__grid {
    display: grid;
    gap: v.$espacio;
    grid-template-columns:
      repeat(auto-fill, minmax(220px, 1fr));
  }
}
```

Desarrollo Web Integrado · Sesión 11 · UTP 11

11

LAB 11 · PASO 10

Paso 10: ensambla la aplicación

src/app/app.ts

```
import { Component } from '@angular/core';
import { Header } from './components/header/header';
import { ProductList }
  from './components/product-list/product-list';

@Component({
  selector: 'app-root',
  imports: [Header, ProductList],
  templateUrl: './app.html',
  styleUrl: './app.scss'
})
export class App {}
```

src/app/app.scss

```
.contenedor { max-width: 1100px;
  margin: 0 auto; padding: 24px; }
```

src/app/app.html (borra todo lo anterior)

```
<app-header />
<main class="contenedor">
  <app-product-list />
</main>
```

Verifica

Guarda todos los archivos. Con ng serve activo, el navegador se actualiza solo y debes ver el header y las 4 tarjetas.

Desarrollo Web Integrado · Sesión 11 · UTP 12

12

LAB 11 · PRUEBAS

Pruebas: ¿tu aplicación funciona?

N.°	Qué hacer	Resultado esperado
1	Ejecutar ng serve	Sin errores; abre localhost:4200
2	Mirar el catálogo	4 tarjetas con nombre, precio en S/ y stock
3	Mirar el Teclado Redragon	Tarjeta atenuada, etiqueta Agotado y botón deshabilitado
4	Pulsar Agregar en una tarjeta	El número entre paréntesis aumenta
5	Leer el aviso de la lista	Muestra “Sin stock: 1”
6	Reducir la ventana a menos de 768 px	El menú del header pasa a columna
7	Cambiar $color-primario en _variables.scss	Todos los elementos rojos cambian al guardar

Desarrollo Web Integrado · Sesión 11 · UTP 13

13

LAB 11 · ERRORES COMUNES

Errores comunes y cómo resolverlos

Síntoma	Causa probable	Solución
'app-product-card' is not a known element	Falta importar el componente	Agregar ProductCard en imports del padre
Can't bind to 'producto'	El hijo no declara el input	Declarar producto = input.required<Producto>()
Can't resolve '../../../scss/variables'	Ruta relativa incorrecta	Revisar niveles de carpeta y que exista _variables.scss
Pantalla en blanco	Error en tiempo de ejecución	Abrir F12, pestaña Console, y leer el mensaje
ng no se reconoce como comando	CLI no instalada o terminal antigua	Instalar el CLI y abrir una terminal nueva
Puerto 4200 ocupado	Otra instancia en ejecución	ng serve --port 4300

Desarrollo Web Integrado · Sesión 11 · UTP 14

14

LAB 11 · RETOS Y ENTREGA

Retos adicionales y evaluación

Retos opcionales

- Agrega un botón “Solo con stock” con un signal y un computed que filtre la lista.
- Crea un tema oscuro cambiando solo las variables SASS.
- Agrega una categoría y dos productos nuevos y verifica el @for.
- Compara tu interface Producto con el JSON de GET /api/productos de tu API Spring Boot.

Criterio de evaluación	Puntos
La aplicación compila y se ejecuta	4
Componentes generados con el CLI y bien nombrados	4
Enlaces de datos, signals y control de flujo	5
SASS: variables, parciales y mixins	5
Orden del código y reto adicional	2
Total	20